

Teach Module 12: Java Coding Standards and Best Practices

This README captures the Module 12 teaching notes, practice exercises, and lab from the chat.

The course document was used only to identify the Module 12 topic list. The teaching content below is independent and not copied from the course material.

Module 12 Overview

Module 12 focuses on writing Java code that is clean, readable, maintainable, and easier for other developers to review.

Good Java code is not only code that works. It should also be clear, predictable, testable, and resistant to avoidable bugs.

Topics Covered

1. Naming conventions
2. Refactoring principles
3. Readability and code clarity
4. Defensive programming
5. Code review practices using IntelliJ
6. Lab: Coding standards and refactoring

1. Naming Conventions

Names should explain intent.

Poor naming:

```
int d;  
String x;  
List<String> l;
```

Better naming:

```
int daysUntilExpiration;  
String customerEmail;  
List<String> activeUsernames;
```

Common Java conventions:

```
class CustomerAccount { }           // Classes: PascalCase  
  
interface PaymentProcessor { }      // Interfaces: PascalCase  
  
void calculateInvoiceTotal() { }    // Methods: camelCase  
  
int retryCount = 3;                 // Variables: camelCase
```

```
static final int MAX_RETRIES = 3; // Constants: UPPER_SNAKE_CASE
```

A good rule: if someone has to ask "what does this mean?", the name is probably too vague.

2. Refactoring Principles

Refactoring means improving code structure without changing behavior.

Before refactoring:

```
public double calculate(double price, int quantity, boolean premium) {
    double total = price * quantity;

    if (premium) {
        total = total * 0.9;
    }

    total = total + 5.99;

    return total;
}
```

After refactoring:

```
public double calculateOrderTotal(double unitPrice, int quantity, boolean premiumCustomer) {
    double subtotal = unitPrice * quantity;
    double discountedSubtotal = applyDiscount(subtotal, premiumCustomer);
    return addShipping(discountedSubtotal);
}

private double applyDiscount(double amount, boolean premiumCustomer) {
    return premiumCustomer ? amount * 0.9 : amount;
}

private double addShipping(double amount) {
    return amount + 5.99;
}
```

The second version is longer, but easier to understand and modify. Each method has one clear job.

3. Readability and Code Clarity

Readable code favors simple logic.

Avoid deep nesting:

```
if (user != null) {
    if (user.isActive()) {
        if (user.hasPermission("ADMIN")) {
            System.out.println("Access granted");
        }
    }
}
```

```
    }  
  }  
}
```

Prefer guard clauses:

```
if (user == null || !user.isActive()) {  
    return;  
}  
  
if (user.hasPermission("ADMIN")) {  
    System.out.println("Access granted");  
}
```

Guard clauses keep the main logic easier to see.

Also avoid clever code when simple code is clearer. In professional Java development, boring and obvious is often a compliment.

4. Defensive Programming

Defensive programming means expecting bad inputs, invalid states, or failures.

Example:

```
public void registerUser(String email) {  
    if (email == null || email.isBlank()) {  
        throw new IllegalArgumentException("Email is required");  
    }  
  
    // continue registration  
}
```

Validate important assumptions early.

Common defensive practices:

```
Objects.requireNonNull(customer, "customer must not be null");
```

```
if (amount.compareTo(BigDecimal.ZERO) < 0) {  
    throw new IllegalArgumentException("Amount cannot be negative");  
}
```

Defensive code protects your program from silently doing the wrong thing.

5. Code Review Practices Using IntelliJ

Useful IntelliJ actions:

- Code > Reformat Code

- Code > Optimize Imports
- Analyze > Inspect Code
- Rename refactoring with Shift + F6
- Extract method with Ctrl + Alt + M
- Find usages with Alt + F7

When reviewing Java code, ask:

1. Is the code easy to understand?
2. Are names meaningful?
3. Is there duplicated logic?
4. Are edge cases handled?
5. Are exceptions meaningful?
6. Is the method doing too many things?
7. Are tests needed or missing?

Mini Practice

Refactor this:

```
public void p(String n, int a) {
    if (n != null) {
        if (a > 0) {
            System.out.println(n + " is " + a + " years old");
        }
    }
}
```

Better version:

```
public void printPersonAge(String name, int age) {
    if (name == null || name.isBlank()) {
        throw new IllegalArgumentException("Name is required");
    }

    if (age <= 0) {
        throw new IllegalArgumentException("Age must be positive");
    }

    System.out.println(name + " is " + age + " years old");
}
```

Key lesson: clean Java code communicates intent clearly, handles bad input responsibly, and is easy to review and change.

Practice Exercises

Exercise 1: Rename for Clarity

Take this code and improve all class, method, and variable names:

```
public class A {  
    public void p(String x, int y) {  
        System.out.println(x + " " + y);  
    }  
}
```

Goal: use meaningful names like `Customer` , `printCustomerSummary` , and `customerName` .

Exercise 2: Refactor a Long Method

Start with a method that does too many things:

```
public void processOrder(String customerEmail, double price, int quantity) {  
    if (customerEmail == null || customerEmail.isBlank()) {  
        System.out.println("Invalid email");  
        return;  
    }  
  
    double total = price * quantity;  
  
    if (total > 100) {  
        total = total * 0.9;  
    }  
  
    System.out.println("Sending email to " + customerEmail);  
    System.out.println("Order total: " + total);  
}
```

Tasks:

- Extract validation into its own method.
- Extract discount calculation into its own method.
- Extract email notification into its own method.
- Rename variables where needed.

Exercise 3: Add Defensive Programming

Improve this method:

```
public double calculateDiscount(double price, int quantity) {  
    return price * quantity * 0.1;  
}
```

Tasks:

- Reject negative prices.
- Reject zero or negative quantity.
- Use clear exception messages.
- Add a few test cases manually or with JUnit.

Exercise 4: Remove Duplicate Code

Refactor this:

```
public void printAdminWelcome(String name) {
    System.out.println("Welcome, " + name);
    System.out.println("Loading dashboard...");
}

public void printUserWelcome(String name) {
    System.out.println("Welcome, " + name);
    System.out.println("Loading dashboard...");
}
```

Goal: remove repeated logic without making the code harder to read.

Exercise 5: Improve Readability with Guard Clauses

Refactor nested logic:

```
public void approveLoan(Customer customer) {
    if (customer != null) {
        if (customer.getCreditScore() > 700) {
            if (customer.getIncome() > 50000) {
                System.out.println("Loan approved");
            }
        }
    }
}
```

Goal: make the method flatter and easier to scan.

Exercise 6: Code Review Practice

Review this code as if you are doing a team code review:

```
public class Calc {
    public double c(double a, double b, String t) {
        if (t == "add") {
            return a + b;
        } else if (t == "sub") {
            return a - b;
        }
        return 0;
    }
}
```

Find issues related to:

- Naming
- String comparison
- Error handling
- Default return value

- Maintainability

Exercise 7: IntelliJ Refactoring Practice

In IntelliJ, create a small Java class and practice:

- Rename class with `Shift + F6`
- Rename method with `Shift + F6`
- Extract method with `Ctrl + Alt + M`
- Reformat code
- Optimize imports
- Run code inspection

Exercise 8: Clean Up a Small Java Class

Create a messy class with:

- Poor names
- Duplicate code
- Long methods
- Missing validation
- Inconsistent formatting

Then refactor it into clean, readable Java.

Capstone-Style Practice

Build a small `OrderService` class with methods for:

```
calculateSubtotal()
applyDiscount()
validateOrder()
printReceipt()
```

Then review and refactor it using Module 12 principles: naming, readability, defensive programming, and clean method design.

Module 12 Lab: Coding Standards and Refactoring

Goal

Take messy Java code and improve it using naming conventions, refactoring, readability, defensive programming, and code review habits.

Create a Java file named:

```
OrderService.java
```

Paste this intentionally messy code:

```
public class OrderService {

    public void p(String e, double p, int q, boolean d) {
```

```

    if (e != null) {
        if (!e.equals("")) {
            double t = p * q;
            if (d == true) {
                t = t - (t * 0.10);
            }
            if (q > 0) {
                System.out.println("email sent to " + e);
                System.out.println("total is " + t);
            } else {
                System.out.println("bad quantity");
            }
        } else {
            System.out.println("bad email");
        }
    } else {
        System.out.println("bad email");
    }
}
}

```

Your Tasks

1. Rename the method `p` to something meaningful.
2. Rename variables `e`, `p`, `q`, `d`, and `t`.
3. Add defensive validation for missing email.
4. Add defensive validation for negative price.
5. Add defensive validation for zero or negative quantity.
6. Replace nested `if` statements with guard clauses.
7. Extract smaller methods:
 - `validateOrder`
 - `calculateSubtotal`
 - `applyDiscount`
 - `sendConfirmationEmail`
 - `printOrderTotal`
8. Use exceptions instead of printing validation errors.
9. Reformat the code in IntelliJ.
10. Use IntelliJ's refactoring tools where possible.

Expected Refactored Version

Try it yourself first, then compare with this:

```

public class OrderService {

    public void processOrder(String customerEmail, double unitPrice, int quantity, boolean discountEligible) {
        validateOrder(customerEmail, unitPrice, quantity);

        double subtotal = calculateSubtotal(unitPrice, quantity);
        double total = applyDiscount(subtotal, discountEligible);
    }
}

```



```

        sendConfirmationEmail(customerEmail);
        printOrderTotal(total);
    }

    private void validateOrder(String customerEmail, double unitPrice, int quantity) {
        if (customerEmail == null || customerEmail.isBlank()) {
            throw new IllegalArgumentException("Customer email is required");
        }

        if (unitPrice < 0) {
            throw new IllegalArgumentException("Unit price cannot be negative");
        }

        if (quantity <= 0) {
            throw new IllegalArgumentException("Quantity must be greater than zero");
        }
    }

    private double calculateSubtotal(double unitPrice, int quantity) {
        return unitPrice * quantity;
    }

    private double applyDiscount(double subtotal, boolean discountEligible) {
        if (discountEligible) {
            return subtotal * 0.90;
        }

        return subtotal;
    }

    private void sendConfirmationEmail(String customerEmail) {
        System.out.println("Email sent to " + customerEmail);
    }

    private void printOrderTotal(double total) {
        System.out.println("Total is " + total);
    }
}

```

Bonus Task

Create a `Main.java` file and test it:

```

public class Main {
    public static void main(String[] args) {
        OrderService orderService = new OrderService();

        orderService.processOrder("student@example.com", 50.0, 3, true);
    }
}

```

Expected output:

```
Email sent to student@example.com  
Total is 135.0
```

Code Review Questions

After refactoring, answer these:

1. Are the method names clear?
2. Does each method do one thing?
3. Is invalid input handled early?
4. Is the main method easy to read?
5. Would another developer understand this code quickly?

This lab practices what clean Java work feels like: naming, refactoring, validation, readable control flow, and review thinking.